

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №18

Тема: Створення лінійних алгоритмів

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати структуру «слідування» (лінійний алгоритм), навчитися користуватися змінними, оператором присвоєння, введенням і виведенням даних, цілочисельними операціями // та %, розробляти лінійні алгоритми для розв'язання прикладних задач і перевіряти їх таблицею трасування та тестами.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Лінійний алгоритм — це алгоритм, у якому всі команди виконуються послідовно, одна за одною, рівно по одному разу, без перевірки умов і повторень. Типова схема лінійного алгоритму: **введення даних** → **обчислення** → **виведення результату**.

2. Змінна та оператор присвоєння. Змінна — іменована область пам'яті для зберігання значення. Оператор присвоєння $x = \text{вираз}$ спочатку обчислює праву частину, а потім записує результат у змінну, тому запис $s = s + 5$ коректний (нове значення s обчислюється зі старого). Імена змінних повинні відображати їхній зміст (`price`, `total`, а не `a1`, `q`).

3. Введення й виведення в Python. Функція `input()` повертає **рядок**, тому для обчислень його потрібно перетворити: `int(input())` — ціле число, `float(input())` — дійсне. Виведення виконує `print()`; для зручного форматування використовують f-рядки: `print(f"Площа = {s:.2f}")` (два знаки після коми).

4. Арифметичні операції для цілих чисел:

Операція	Запис	Приклад	Результат
Ділення націло	$a // b$	<code>47 // 10</code>	4
Залишок від ділення	$a \% b$	<code>47 \% 10</code>	7
Звичайне ділення	a / b	<code>47 / 10</code>	4.7
Піднесення до степеня	$a ** b$	<code>2 ** 10</code>	1024
Модуль числа	<code>abs(a)</code>	<code>abs(-5)</code>	5

Операції `//` і `%` — основа багатьох лінійних алгоритмів: виділення цифр числа ($n \% 10$ — остання цифра, $n // 10$ — число без останньої цифри), переведення одиниць виміру (секунди → хвилини), обчислення кількості повних груп.

5. Обмін значень двох змінних. Щоб обміняти значення змінних a і b , використовують допоміжну змінну: `t = a; a = b; b = t`. У Python допустимий також кортежний запис `a, b = b, a`.

$b = b, a$. Можливий і арифметичний спосіб без допоміжної змінної: $a = a + b; b = a - b; a = a - b$.

6. Порядок розроблення. Постановка задачі → вхідні та вихідні дані → формули → запис алгоритму → трасування на прикладі → програма → тестування. Перед написанням програми обов'язково виконайте приклад «на папері» – це дозволяє перевірити формули.

Хід роботи

Завдання 1. Трасування: обмін значень і виділення цифр

Виконати вручну два лінійні алгоритми та побудувати таблиці трасування.

```
алг Обмін
  a := 5; b := 9
  a := a + b
  b := a - b
  a := a - b
  вивести a, b
кін
```

```
алг Цифри
  ввести n           {n = 472}
  c := n mod 10
  n := n div 10
  b := n mod 10
  a := n div 10
  вивести a, b, c
кін
```

Порядок виконання:

1. Для алгоритму «Обмін» побудуйте таблицю зі стовпцями «крок», «a», «b»; запишіть значення змінних після кожного оператора.
2. З'ясуйте, що робить цей алгоритм, і поясніть, чому для обміну не знадобилась допоміжна змінна.
3. Для алгоритму «Цифри» (mod — залишок, div — ділення націло) виконайте трасування для $n = 472$: запишіть значення n, a, b, c після кожного кроку.
4. Сформулюйте, що обчислює алгоритм «Цифри» (яка цифра в якій змінній опиняється).
5. Повторіть трасування алгоритму «Цифри» для $n = 908$; поясніть, чому другий крок дає $b = 0$, і чи залишається алгоритм коректним.

Завдання 2. Складання лінійного алгоритму для прикладної задачі

Задача «Час у дорозі». Потяг проїхав відстань s кілометрів із середньою швидкістю v км/год. Визначити час у дорозі в годинах і хвилинах та вартість поїздки, якщо квиток коштує c грн за 100 км.

Порядок виконання:

1. Запишіть постановку задачі: вхідні дані (s, v, c), вихідні (години, хвилини, вартість).
2. Виведіть формули: повний час у хвилинах $t = s / v \cdot 60$ (округліть до цілих хвилин), години $= t // 60$, хвилини $= t \% 60$, вартість $= s / 100 \cdot c$.
3. Складіть псевдокод і блок-схему алгоритму (лише фігури введення, обчислення, виведення).
4. Виконайте трасування для $s = 450, v = 90, c = 35$: запишіть проміжні значення t, h, m та вартості.
5. Виконайте трасування для $s = 500, v = 80$ та порівняйте з очікуваним результатом 6 год 15 хв.
6. Запишіть, яка з величин у цьому алгоритмі — цілого, а яка — дійсного типу, і чому.

Завдання 3. Програмна реалізація лінійних алгоритмів

Реалізувати мовою Python кілька лінійних алгоритмів у вигляді функцій і перевірити їх тестами.

Порядок виконання:

1. Напишіть функцію `swap_no_temp(a, b)`, що обмінює значення двох чисел арифметичним способом (за алгоритмом із завдання 1) і повертає пару (a, b).
2. Напишіть функцію `digits3(n)`, що розбиває тризначне число на цифри за допомогою `//` та `%` і повертає кортеж (сотні, десятки, одиниці); додайте до неї функцію `digit_sum(n)` — суму цифр.
3. Напишіть функцію `travel(s, v, c)` для задачі «Час у дорозі», що повертає кортеж (години, хвилини, вартість).
4. Напишіть функцію `seconds_to_hms(sec)`, що переводить кількість секунд у формат «г:хв:с» (наприклад, 3725 с = 1:02:05).
5. Напишіть функцію `distance(x1, y1, x2, y2)` – відстань між двома точками на площині за формулою $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$ (використайте `math.sqrt` або піднесення до степеня 0.5).
6. Перевірте кожну функцію на 2–3 тестах (у тому числі для даних із ручного трасування); результати виведіть у форматованому вигляді з потрібною кількістю знаків після коми.

Приклад реалізації мовою Python:

python

```
import math

def swap_no_temp(a, b):
    a = a + b
    b = a - b
    a = a - b
    return a, b

def digits3(n):
    hundreds = n // 100
    tens = n // 10 % 10
    units = n % 10
    return hundreds, tens, units

def digit_sum(n):
    h, t, u = digits3(n)
    return h + t + u

def travel(s, v, c):
    minutes = round(s / v * 60)
    return minutes // 60, minutes % 60, s / 100 * c

def seconds_to_hms(sec):
    h = sec // 3600
    m = sec % 3600 // 60
    s = sec % 60
    return f"{h}:{m:02d}:{s:02d}"

def distance(x1, y1, x2, y2):
    return math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
```

```

print("Обмін (5, 9) ->", swap_no_temp(5, 9))
print("Цифри 472 ->", digits3(472), ", сума цифр =", digit_sum(472))
print("Цифри 908 ->", digits3(908), ", сума цифр =", digit_sum(908))
h, m, cost = travel(450, 90, 35)
print(f"Потяг: {h} год {m} хв, вартість {cost:.2f} грн")
h, m, cost = travel(500, 80, 35)
print(f"Потяг: {h} год {m} хв, вартість {cost:.2f} грн")
print("3725 с =", seconds_to_hms(3725))
print("86399 с =", seconds_to_hms(86399))
print(f"Відстань (1, 2)-(4, 6) = {distance(1, 2, 4, 6):.2f}")
print(f"Відстань (0, 0)-(1, 1) = {distance(0, 0, 1, 1):.4f}")

```

Результат виконання програми:

```

Обмін (5, 9) -> (9, 5)
Цифри 472 -> (4, 7, 2) , сума цифр = 13
Цифри 908 -> (9, 0, 8) , сума цифр = 17
Потяг: 5 год 0 хв, вартість 157.50 грн
Потяг: 6 год 15 хв, вартість 175.00 грн
3725 с = 1:02:05
86399 с = 23:59:59
Відстань (1, 2)-(4, 6) = 5.00
Відстань (0, 0)-(1, 1) = 1.4142

```

Висновок за результатами: результати програми збігаються з ручним трасуванням: числа 5 і 9 обмінялись, число 472 розкладене на цифри (4, 7, 2), потяг за умовами $s = 500$, $v = 80$ їхав 6 год 15 хв. Завдяки операціям `//` і `%` цифри числа та складові часу виділяються без циклів і розгалужень — саме тому такі алгоритми лінійні. Формат 15 додає ведучий нуль, а 175.00 обмежує дійсне число двома знаками після коми.

Завдання 4. Розширені лінійні задачі та дослідження типів даних

Розв'язати кілька додаткових лінійних задач і дослідити, як тип даних впливає на результат обчислень.

Порядок виконання:

1. Напишіть функцію `celsius_to_fahrenheit(c)` за формулою $F = C \cdot 9/5 + 32$ і виведіть таблицю для значень -40 , 0 , 36.6 , 100 (зверніть увагу: при -40 значення шкал збігаються).
2. Напишіть функцію `pocket(total, price)`, що повертає, скільки повних товарів ціною `price` можна купити на суму `total` та яка решта залишиться (використайте `//` та `%`).
3. Напишіть функцію `circle(r)`, що повертає довжину кола та площу круга ($\pi = \text{math.pi}$).
4. Напишіть функцію `reverse3(n)`, що «перевертає» тризначне число ($472 \rightarrow 274$), використовуючи лише арифметичні операції, без перетворення в рядок.
5. Порівняйте результати `7 / 2`, `7 // 2`, `7 % 2`, `-7 // 2`, `-7 % 2`, `int("12") + 1`, `"12" + "1"`, `type(7 / 2)`; поясніть кожен результат.
6. Поясніть, чому значення, прочитане функцією `input()`, не можна одразу використовувати в обчисленнях, і як саме це виправляється.

Приклад реалізації мовою Python:

python

```

import math

def celsius_to_fahrenheit(c):

```

```

    return c * 9 / 5 + 32

def pocket(total, price):
    return total // price, total % price

def circle(r):
    return 2 * math.pi * r, math.pi * r ** 2

def reverse3(n):
    a, b, c = n // 100, n // 10 % 10, n % 10
    return c * 100 + b * 10 + a

for c in (-40, 0, 36.6, 100):
    print(f"{c:>6} °C = {celsius_to_fahrenheit(c):>6.1f} °F")

count, change = pocket(250, 35)
print(f"На 250 грн при ціні 35 грн: куплено {count} шт., решта {change} грн")

length, area = circle(3)
print(f"Коло r = 3: довжина = {length:.3f}, площа = {area:.3f}")
print("reverse3(472) =", reverse3(472), "; reverse3(120) =", reverse3(120))

print("7 / 2 =", 7 / 2, type(7 / 2).__name__)
print("7 // 2 =", 7 // 2, type(7 // 2).__name__)
print("7 % 2 =", 7 % 2)
print("-7 // 2 =", -7 // 2, "; -7 % 2 =", -7 % 2)
print("int('12') + 1 =", int("12") + 1, "; '12' + '1' =", "12" + "1")

```

Результат виконання програми:

```

    -40 °C =  -40.0 °F
      0 °C =   32.0 °F
   36.6 °C =   97.9 °F
   100 °C =  212.0 °F
На 250 грн при ціні 35 грн: куплено 7 шт., решта 5 грн
Коло r = 3: довжина = 18.850, площа = 28.274
reverse3(472) = 274 ; reverse3(120) = 21
7 / 2 = 3.5 float
7 // 2 = 3 int
7 % 2 = 1
-7 // 2 = -4 ; -7 % 2 = 1
int('12') + 1 = 13 ; '12' + '1' = 121

```

Висновок за результатами: при $-40\text{ }^{\circ}\text{C}$ і за Цельсієм, і за Фаренгейтом отримуємо однакове значення (-40.0), що є зручним контрольним тестом формули. Результат `reverse3(120)` дорівнює 21, тобто ведучий нуль «зникає» — це наслідок роботи з числами, а не з рядками, і про такий граничний випадок слід пам'ятати. Операція `/` завжди дає тип `float`, тоді як `//` для цілих операндів дає `int`; для від'ємних чисел `//` округлює донизу ($-7 // 2 = -4$), а залишок `%` при цьому невід'ємний. Додавання рядків `'12' + '1'` виконує конкатенацію (`'121'`), а не арифметичне додавання, тому значення з `input()` потрібно перетворювати функціями `int()` або `float()`.

Контрольні питання

1. Який алгоритм називається лінійним? З яких етапів складається типовий лінійний алгоритм?
2. Що таке змінна? Як працює оператор присвоєння? Чому запис $s = s + 5$ коректний?
3. Який тип даних повертає функція `input()` і як перетворити введене значення на число?
4. Яка різниця між операціями `/`, `//` і `%`? Наведіть приклади.
5. Як виділити останню цифру числа та відкинути її? Як отримати середню цифру тризначного числа?
6. Назвіть способи обміну значень двох змінних. У чому перевага допоміжної змінної, а в чому — арифметичного способу?
7. Як переводити секунди в години, хвилини та секунди за допомогою `//` та `%`?
8. Для чого при розробленні алгоритму виконують ручне трасування до написання програми?

Зміст звіту

1. Тема, мета роботи.
2. Таблиці трасування та розрахунки (завдання 1, 2), блок-схема або псевдокод до завдання 2.
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Відповіді на питання завдання 4 про поведінку операцій і типів даних.
5. Висновки.